The Journal of Open Source Software

DeLTA 3.0: A modern segmentation and tracking tool for microscopy images

Virgile Andreani [1,2¶], **Jean-Baptiste Lugagne** [1,3], **Owen M. O'Connor** [1], **and Mary J. Dunlop** [1¶]

1 Biomedical Engineering, Boston University, United States of America **2** EPI Lifeware, Inria Saclay, France **3** Department of Engineering Science, University of Oxford, United Kingdom ¶ Corresponding author

DOI: 10.21105/joss.10279

Software

- Review
- Repository
- Archive

Editor: David Stansby

Reviewers:

- @msschwartz21
- @JojoDevel

Submitted: 10 October 2025
Published: 16 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Time-lapse microscopy is a popular approach for quantifying single-cell dynamics. Analyzing image sequences requires two steps: segmentation (locating the cells) and tracking (following them). DeLTA (Deep Learning for Time-lapse Analysis) is a Python package that performs segmentation and tracking. Initially developed to handle images of bacteria growing within the "mother machine," (Wang et al., 2010) a popular microfluidic device for single-cell analysis that constrains cells to grow in one dimension (Lugagne et al., 2020), it was later extended to handle cells growing in 2D (O'Connor et al., 2022). After three years of experience and feedback, we are releasing version 3.0 — a major overhaul to achieve four objectives: enhance usability for scientists, improve modularity and adaptability for developers, integrate features like growth rate computation and lineage manipulation, and adopt modern open-source development practices.

Statement of need

DeLTA is a deep learning pipeline for analyzing bacterial time-lapse microscopy images.

This version adds functionality while remaining true to DeLTA's principles. DeLTA distinguishes itself by:

- its built-in handling of mother machine settings, a context where segmentation and tracking are performed within several regions of interest on each frame;
- its ability to cater to novice and experienced users alike, respectively through a new interface allowing users to run and train DeLTA without writing code, and through an API allowing programmatic and real-time use;
- new utilities provided for common analysis functions (e.g., robust estimation of growth rate, correcting tracking errors).

Additionally, in v3.0, deep learning models have been ported to Keras, a high-level backend-agnostic library that allows users to select any of three deep learning backends (TensorFlow (Abadi et al., 2015), PyTorch (Ansel et al., 2024), or JAX).

State of the field

When DeLTA was originally developed, only CellProfiler (Stirling et al., 2021) and DeepCell (Valen et al., 2016) existed in Python for cell segmentation and tracking with deep learning. Other tools have since been developed, including StarDist (Schmidt et al., 2018; Weigert et al., 2020) (limited to the segmentation of star-convex shapes, not appropriate for filamenting

cells), Cellpose (Stringer et al., 2021), Omnipose (Cutler et al., 2022), MM3 (Thiermann et al., 2024), Trackastra (Gallusser & Weigert, 2025), and Cell-TRACTR (O'Connor & Dunlop, 2025). DeLTA's first version was developed for the mother machine. To date, only DeLTA and MM3 offer built-in mother machine support. The second version of DeLTA extended analysis to two-dimensional growth. The current version makes it, to our knowledge, the only package to also provide robust growth rate estimation and utilities to manipulate lineages.

Software design

DeLTA's design philosophy is based on a trade-off around four principles: (1) user-friendly defaults and interfaces to empower less-technical users, (2) a modular API to allow expert users to address all situations, (3) minimizing custom code when reliable and performant libraries exist, and (4) providing reliable utilities for the most common needs of users beyond segmentation and tracking.

Version 3 aims to improve all four principles:

1. The new command-line interface enables using DeLTA without writing code for segmentation, tracking, and model training.
2. The API was rationalized to reduce naming and usage inconsistencies.
3. We adopted an open data model, based on `xarray` (in memory) and `netCDF4` (for file storage), to replace our custom `pickle`-based object serialization. See *File format of results* for details and trade-offs of this approach.
4. We provided two utilities to address common needs of experimentalists: the computation of instantaneous cell growth rate, and an interface to fix tracking errors. See the *Growth rate* and *Lineage* sections for details on implementation.

Research impact statement

As one of the first packages using deep learning for time-lapse analysis, DeLTA has had significant research impact (Ahmadi et al., 2024; Allard et al., 2022; Gericke et al., 2024) and is an inspiration or reference point for the development of other packages (O'Connor & Dunlop, 2025; Thiermann et al., 2024).

DeLTA is widely used by researchers internationally. Currently, DeLTA is downloaded ~220 times per month on the Python Packaging Index (PyPI) and ~430 times per month on conda-forge.

General principles of the package

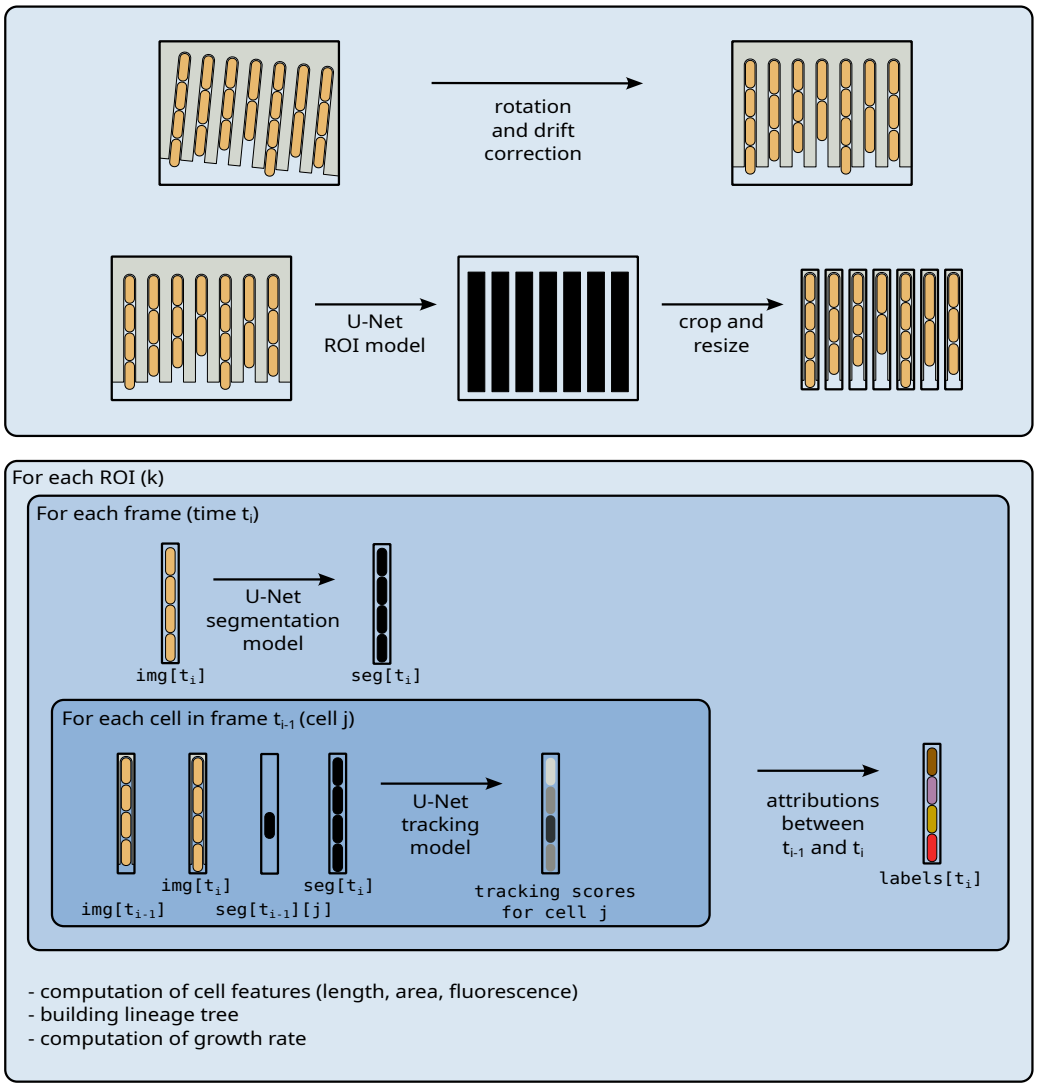


Figure 1: Schematic principles of DeLTA. First, rotation and drift are corrected. Then, for mother machine experiments, a ROI model detects individual chambers. Then, ROI frames are segmented, and passed to a tracking model. Finally, morphological cell features are computed, the lineage tree is built, and growth rates are computed.

DeLTA uses three different U-Net (Ronneberger et al., 2015) models: for region of interest (ROI) identification, segmentation, and tracking (figure 1). Once ROIs have been extracted, the segmentation model is run frame-by-frame to determine the location of cells. Finally, for every pair of consecutive frames, the tracking model is run once per cell in the earlier frame, to determine the probabilities that this cell corresponds to each cell in the later frame. After mother-daughter relationships have been determined, a lineage tree is built (see the *Lineage* section) and cell features extracted: area, length, average fluorescence, and growth rate (*Growth rate* section). Finally, a netCDF file and segmented movie are saved for analysis and visualization.

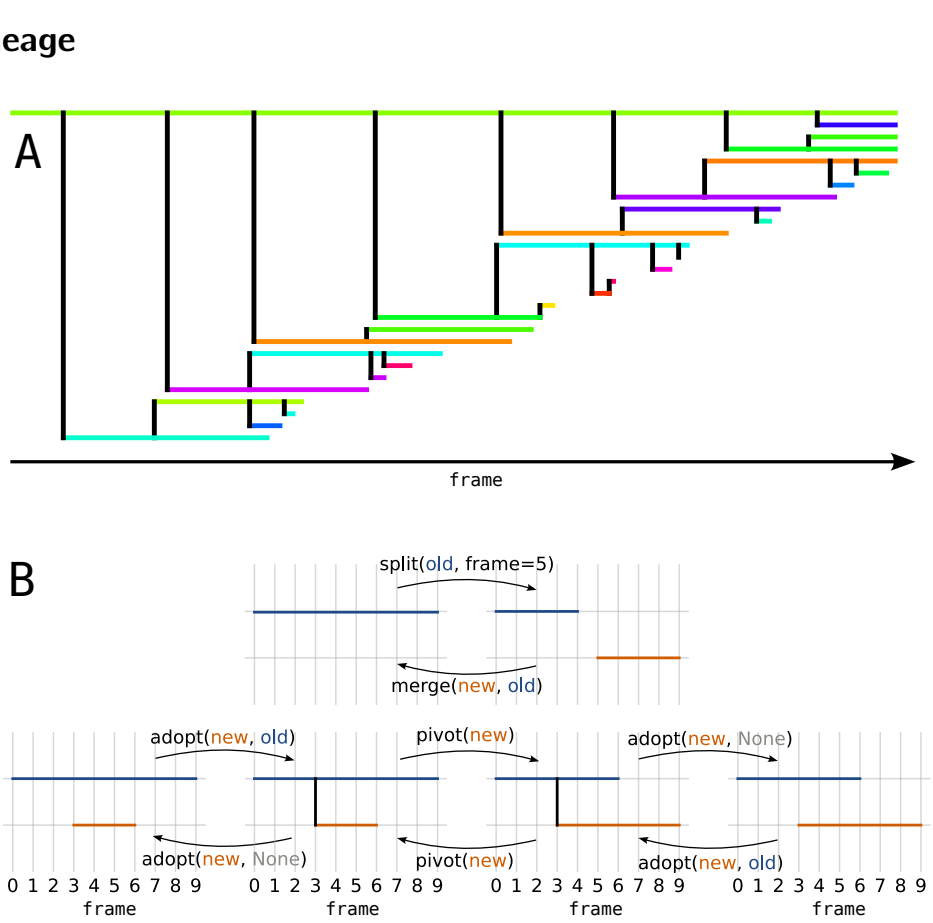


Figure 2: Lineage tree manipulation. **A**: Lineage tree of one mother machine chamber. Horizontal lines represent individual cells, and vertical lines division events. Cells exit the mother machine, so some lineages terminate before the end of the figure. The top cell is trapped and remains forever in the device. **B**: These functions are used to correct tracking errors. The `merge` and `split` functions can be used to mend or break a lineage. The `adopt` function can set or unset a mother-daughter relationship, while `pivot` swaps mother and daughter.

DeLTA detects cell divisions, linking mother and daughter cells. Bacterial cell divisions are typically symmetrical, however pole age has measurable effects on cell physiology, where the daughter cell that inherits the older end of the mother cell grows more slowly than the other daughter (Stewart et al., 2005). For this reason, we consider that the mother cell persists through divisions, giving birth to one daughter (instead of the mother cell disappearing and giving rise to two daughter cells). Thus, upon cell division, the mother cell retains its ID number, while the new cell is given a new ID.

DeLTA builds the lineage tree frame-by-frame, extending existing cells and creating new daughter cells at divisions. However, tracking errors are possible. These cases can be corrected by relabeling the cells involved. However, relabeling requires updating an entire lineage subtree, an error-prone operation. For this reason, we implemented a set of elementary operations on the tree, whose combined effect allows for arbitrarily complex lineage tree manipulations (figure 2).

Growth rate

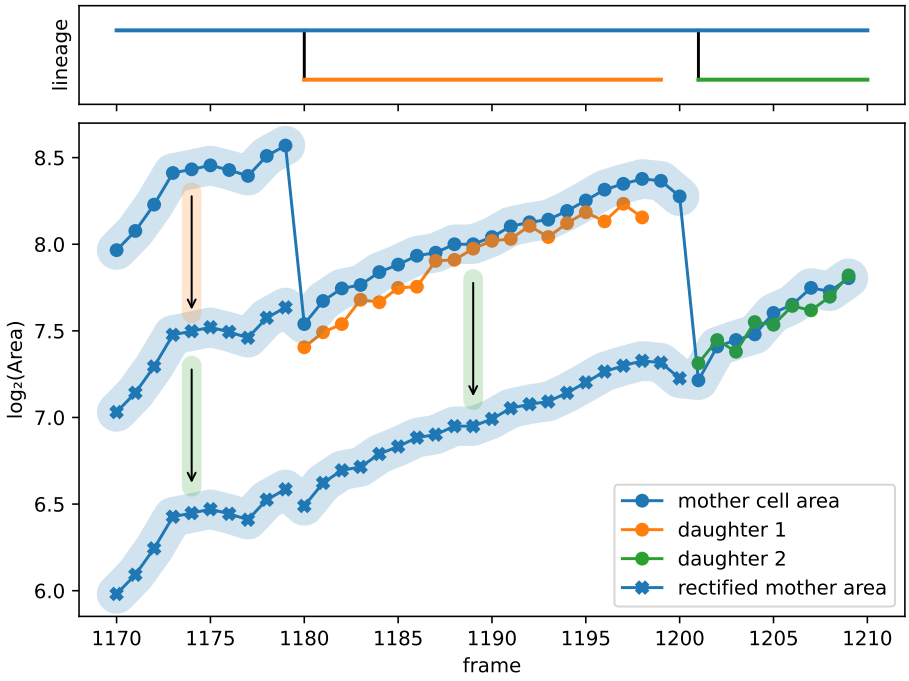


Figure 3: Alignment of cell areas for growth rate computation. Circle markers show the logarithm of the area (number of pixels) of a mother cell and two of its daughters. Jumps are where cells divide. Crosses show entire cell cycles that are offset (colored arrows) as a function of the relative sizes of the cells after division (highlighted pair of points). The offset is $\log_2\left(1+\frac{\text{Area}_{\text{mother}}}{\text{Area}_{\text{daughter}}}\right)$. After offsetting, the growth rate can be computed everywhere with a centered difference scheme.

Single-cell growth rate is considered a proxy for cell fitness. Within a population, individual cells have different growth rates, fluctuating with time. The single-cell growth rate, $\mu(t)$, satisfies the following differential equation with $\ell(t)$ the cell length:

$$\frac{\mathrm{d}\ell}{\mathrm{d}t} = \mu(t)\ell(t)$$

DeLTA can calculate growth rates based on cell area or length. Area-based growth rates rely on direct counts of the number of pixels per cell, while length-based growth rates derive length estimates from the bacterial shape. Growth rates based on volume cannot be calculated with images at a single focal plane.

Instantaneous growth rate is computed in DeLTA as the derivative of the logarithm of cell length (or area). To limit segmentation noise, we use a Savitzky-Golay filter (Savitzky & Golay, 1964) to simultaneously smooth and differentiate the signal. It is a centered finite difference scheme: a deliberate choice to avoid discretization biases.

To compute the growth rate, the Savitzky-Golay filter requires one data point from the previous frame and one from the next. To avoid jumps in growth rate around cell divisions, we offset the areas before and after division by extrapolation of the lineage. This amounts to considering that the area of a mother cell after division is the sum of its own area and that of its daughter (figure 3).

File format of results

Analysis results produce image-like data (segmentation masks and label masks), and scalar data for every cell (ID of mother cell), or every cell and frame (features such as length, area, average fluorescence, growth rate). In DeLTA 2.0 we used the `pickle` library to serialize the results into a file. While this allowed for saving arbitrary Python objects, these files could only be opened with the same version of DeLTA that created them.

In version 3 we now use netCDF4 (Rew et al., 1989), an open file format based on HDF5 (Group, 2025), popular for storing scientific data. A limitation of this approach, compared to pickle files, is that the HDF5 data format only supports collections of rectangular arrays, which required us to reorganize cell features and lineages in a rectangular fashion, at the expense of file size, although this can be mitigated by compression or by using sparse arrays, which is a future goal. The change to netCDF4 is an important improvement for compatibility of result files across future versions of DeLTA.

Conclusion

DeLTA 3.0 enhances function for both novice and experienced users with new features (including command-line interface, lineage tree manipulation and growth rate calculation), better interoperability (results stored as netCDF files), and backend-agnostic deep-learning models (implemented in pure Keras).

Acknowledgements

We acknowledge contributions of Idris Kempf, Caroline Blassick, and Eric Bueno; feedback from the rest of the Dunlop lab; and input from the wider community.

This work was supported by the National Science Foundation (MCB-2143289) to MJD. VA acknowledges support from the European Research Council (ERC-2022-STG, BridgingScales, grant agreement 101075989).

DeLTA would not be the same without our dependencies (BioIO Contributors, 2026; Costa-Luis et al., 2026; Harris et al., 2020; Hoyer & Hamman, 2017; Hunter, 2007; Rew et al., 1989; Tulder & Fringeli, 2024; Ueda et al., 2020; Virtanen et al., 2020; Walt et al., 2014).

AI Usage Disclosure

No generative AI was used in software creation, documentation, or paper authoring.

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jozefowicz, R., Jia, Y., Kaiser, L., Kudlur, M., … Zheng, X. (2015). *TensorFlow, Large-scale machine learning on heterogeneous systems*. https://doi.org/10.5281/zenodo.4724125

Ahmadi, A., Courtney, M., Ren, C., & Ingalls, B. (2024). A benchmarked comparison of software packages for time-lapse image processing of monolayer bacterial population dynamics. *Microbiology Spectrum*. https://doi.org/10.1128/spectrum.00032-24

Allard, P., Papazotos, F., & Potvin-Trottier, L. (2022). Microfluidics for long-term single-cell time-lapse microscopy: Advances and applications. *Frontiers in Bioengineering and Biotechnology*, *10*. https://doi.org/10.3389/fbioe.2022.968342

Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., … Chintala, S. (2024, April). PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. https://doi.org/10.1145/3620665.3640366

BioIO Contributors. (2026). *BioIO* (Version v3.5.1). Zenodo. https://doi.org/10.5281/ZENODO.22755965

Costa-Luis, C. da, Larroque, S. K., Altendorf, K., Mary, H., Richardsheridan, Korobov, M., Raphael, N., Ivanov, I., Bargull, M., Rodrigues, N., Shawn, Dektiarev, M., Górny, M., Mjstevens777, Pagel, M. D., Zugnoni, M., CrazyPython, Newey, C., Lee, A., … Rothberg, A. (2026). *Tqdm: A fast, extensible progress bar for Python and CLI* (Version v4.70.0). Zenodo. https://doi.org/10.5281/ZENODO.595120

Cutler, K. J., Stringer, C., Lo, T. W., Rappez, L., Stroustrup, N., Brook Peterson, S., Wiggins, P. A., & Mougous, J. D. (2022). Omnipose: A high-precision morphology-independent solution for bacterial cell segmentation. *Nature Methods*, *19*(11), 1438–1448. https://doi.org/10.1038/s41592-022-01639-4

Gallusser, B., & Weigert, M. (2025). TRACKASTRA: Transformer-based cell tracking for live-cell microscopy. In A. Leonardis, E. Ricci, S. Roth, O. Russakovsky, T. Sattler, & G. Varol (Eds.), *Computer vision – ECCV 2024* (Vol. 15134, pp. 467–484). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-73116-7_27

Gericke, B., Degner, F., Hüttmann, T., Werth, S., & Fortmann-Grote, C. (2024). Performance review of retraining and transfer learning of DeLTA2 for image segmentation for pseudomonas fluorescens SBW25: *Proceedings of the 17th International Joint Conference on Biomedical Engineering Systems and Technologies*, 273–280. https://doi.org/10.5220/0012316300003657

Group, T. H. (2025). *Hierarchical data format, version 5* (Version 2.0.0). Zenodo. https://doi.org/10.5281/ZENODO.17808614

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hoyer, S., & Hamman, J. (2017). Xarray: N-d labeled arrays and datasets in Python. *Journal of Open Research Software*, *5*(1). https://doi.org/10.5334/jors.148

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Lugagne, J.-B., Lin, H., & Dunlop, M. J. (2020). DeLTA: Automated cell segmentation, tracking, and lineage reconstruction using deep learning. *PLOS Computational Biology*, *16*(4), e1007673. https://doi.org/10.1371/journal.pcbi.1007673

O'Connor, O. M., Alnahhas, R. N., Lugagne, J.-B., & Dunlop, M. J. (2022). DeLTA 2.0: A deep learning pipeline for quantifying single-cell spatial and temporal dynamics. *PLOS Computational Biology*, *18*(1), e1009797. https://doi.org/10.1371/journal.pcbi.1009797

O'Connor, O. M., & Dunlop, M. J. (2025). Cell-TRACTR: A transformer-based model for end-to-end segmentation and tracking of cells. *PLOS Computational Biology*, *21*(5), e1013071. https://doi.org/10.1371/journal.pcbi.1013071

Rew, R., Davis, G., Emmerson, S., Cormack, C., Caron, J., Pincus, R., Hartnett, E., Heimbigner,

D., Appel, L., & Fisher, W. (1989). *Unidata NetCDF*. UCAR/NCAR - Unidata. https://doi.org/10.5065/D6H70CW6

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In N. Navab, J. Hornegger, W. M. Wells, & A. F. Frangi (Eds.), *Medical image computing and computer-assisted intervention – MICCAI 2015* (Vol. 9351, pp. 234–241). Springer International Publishing. https://doi.org/10.1007/978-3-319-24574-4_28

Savitzky, Abraham., & Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, *36*(8), 1627–1639. https://doi.org/10.1021/ac60214a047

Schmidt, U., Weigert, M., Broaddus, C., & Myers, G. (2018). Cell detection with star-convex polygons. In A. F. Frangi, J. A. Schnabel, C. Davatzikos, C. Alberola-López, & G. Fichtinger (Eds.), *Medical image computing and computer assisted intervention – MICCAI 2018* (Vol. 11071, pp. 265–273). Springer International Publishing. https://doi.org/10.1007/978-3-030-00934-2_30

Stewart, E. J., Madden, R., Paul, G., & Taddei, F. (2005). Aging and death in an organism that reproduces by morphologically symmetric division. *PLOS Biology*, *3*(2), e45. https://doi.org/10.1371/journal.pbio.0030045

Stirling, D. R., Swain-Bowden, M. J., Lucas, A. M., Carpenter, A. E., Cimini, B. A., & Goodman, A. (2021). CellProfiler 4: Improvements in speed, utility and usability. *BMC Bioinformatics*, *22*(1), 1–11. https://doi.org/10.1186/s12859-021-04344-9

Stringer, C., Wang, T., Michaelos, M., & Pachitariu, M. (2021). Cellpose: A generalist algorithm for cellular segmentation. *Nature Methods*, *18*(1), 100–106. https://doi.org/10.1038/s41592-020-01018-x

Thiermann, R., Sandler, M., Ahir, G., Sauls, J. T., Schroeder, J., Brown, S., Le Treut, G., Si, F., Li, D., Wang, J. D., & Jun, S. (2024). Tools and methods for high-throughput single-cell imaging with the mother machine. *eLife*, *12*, RP88463. https://doi.org/10.7554/eLife.88463

Tulder, G. van, & Fringeli, G. (2024). *Gvtulder/elasticdeform: Version v0.5.1*. Zenodo. https://doi.org/10.5281/zenodo.11267397

Uieda, L., Soler, S. R., Rampin, R., Kemenade, H. van, Turk, M., Shapero, D., Banihirwe, A., & Leeman, J. (2020). Pooch: A friend to fetch your data files. *Journal of Open Source Software*, *5*(45), 1943. https://doi.org/10.21105/joss.01943

Valen, D. A. V., Kudo, T., Lane, K. M., Macklin, D. N., Quach, N. T., DeFelice, M. M., Maayan, I., Tanouchi, Y., Ashley, E. A., & Covert, M. W. (2016). Deep learning automates the quantitative analysis of individual cells in live-cell imaging experiments. *PLOS Computational Biology*, *12*(11), e1005177. https://doi.org/10.1371/journal.pcbi.1005177

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … Mulbregt, P. van. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Walt, S. van der, Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., & Yu, T. (2014). Scikit-image: Image processing in Python. *PeerJ*, *2*, e453. https://doi.org/10.7717/peerj.453

Wang, P., Robert, L., Pelletier, J., Dang, W. L., Taddei, F., Wright, A., & Jun, S. (2010). Robust growth of escherichia coli. *Current Biology*, *20*(12), 1099–1103. https://doi.org/10.1016/j.cub.2010.04.045

Weigert, M., Schmidt, U., Haase, R., Sugawara, K., & Myers, G. (2020). Star-convex polyhedra for 3D object detection and segmentation in microscopy. *2020 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 3655–3662. https://doi.org/10.1109/WACV45572.2020.9093435